Machine Learning

30-04-26

Lasso regularization

$L = mse + \lambda \sum |w|$

- in some cases weights of less important features become exactly zero

  $w_i = 0$

- zero weights are possible

Ridge regularization

$L = mse + \lambda \sum w^2$

- weights of less important features become very small and closer to zero

  $w_i \approx 0$

- zero weights are nearly impossible

Multicolinearity Issue

- this issue occurs when some features are dependent on each other or if some features are two version of exactly the same feature

eg. house price prediction

area (sqft)	area (sqm)	rooms	pincode	locality	price
⋮	⋮	⋮	0001	01	
⋮	⋮	⋮	0016	16	
⋮	⋮	⋮	0012	12	
⋮	⋮	⋮	⋮	⋮	

same column different units

same column diff format

In above example, w_1 w_2 / w_3 w_4 are trying to optimize same feature

w_1 w_2 → area (sqft) = area (sqm)

w_3 w_4 → pincodes locality

this type of setup makes model training unstable.

solution: regularization

- Regularization will make one of the w_1 w_2 and one of the w_3 w_4 very less → closer to zero and this will rule out their features
- After this model will only optimize on remaining features.

Random Forest

Random → uses random sampling

Forest → combination of multiple decision trees

- supervised ML
- used for regression and classification
- uses a technique called <u>bagging</u>

Bagging → Bootstrap sampling + Aggregation

we have 'n' training datapoints

$d_1 \quad d_2 \quad d_3 \ldots d_n$

Bootstrap sampling is selecting 'n' datapoints randomly from available data with replacement.

for n=5

$d_1 \quad d_2 \quad d_3 \quad d_4 \quad d_5$

1st selection = d_3 $(d_1 \quad d_2 \quad d_3 \quad d_4 \quad d_5)$

2nd selection = d_1 $(d_1 \quad d_2 \quad d_3 \quad d_4 \quad d_5)$

3rd selection = d_3 $(d_1 \quad d_2 \quad d_3 \quad d_4 \quad d_5)$

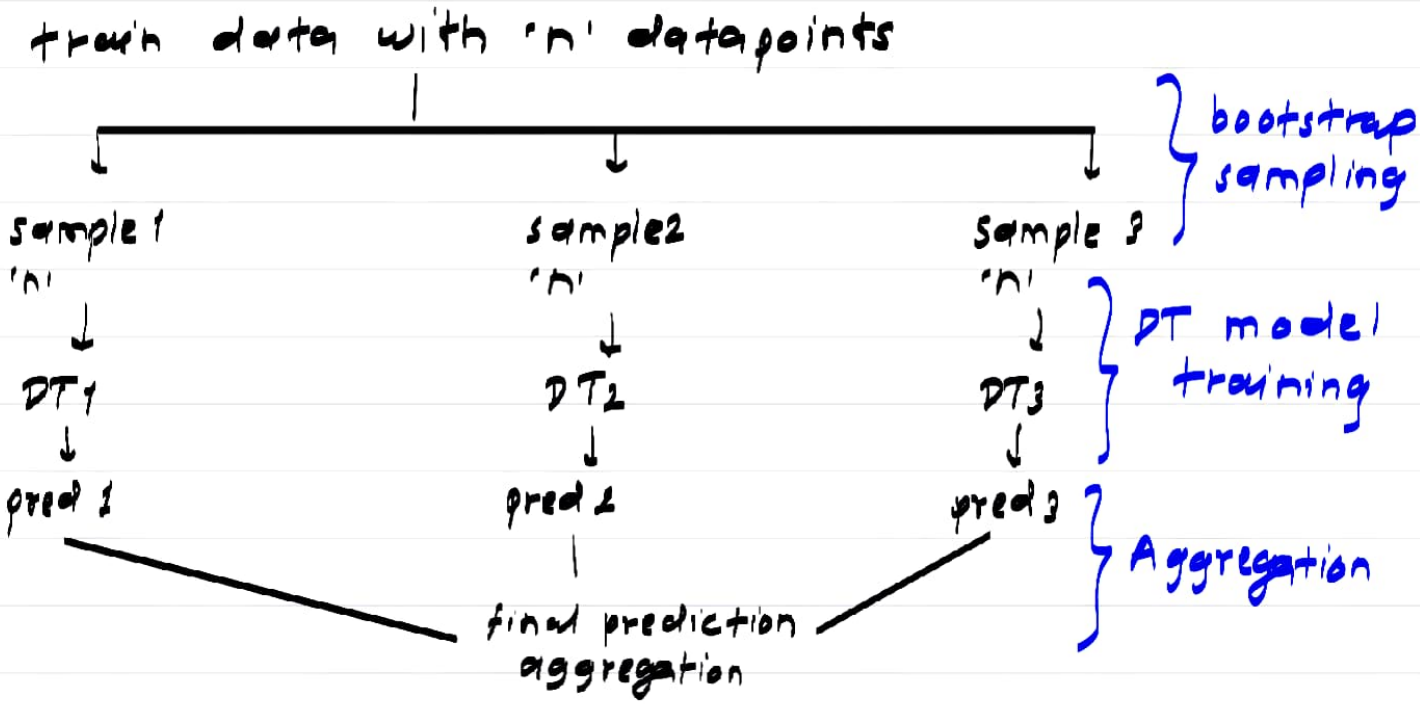
4th selection = d_1 $(d_1 \quad d_2 \quad d_3 \quad d_4 \quad d_5)$

5th selection = d_5 $(d_1 \quad d_2 \quad d_3 \quad d_4 \quad d_5)$

In RF, we decide how many decision trees to train

num of trees → hyperparameter in RF

Lets say we decided to train 3 DTrees in given RF, we'll create 3 bootstrap samples - 1 sample per DTree



- each DT model is trained on one bootstrapped sample
- finally predictions of all DT models are aggrigated together

Total hyperparameters in RF

1) num of trees = n_estimators

2) As we are training multiple DTrees in RF, hyperparameters of DTree will also need to be setup

* Hyperparameter values will be same across all DTrees in a RF model.

RF Regression :

- this will include 'm' DT regression models
- 'm' models generate 'm' predictions

$p_1 \quad p_2 \quad p_3 \ldots\ldots p_m \longrightarrow$ continuous values

final RF prediction $= \frac{1}{m} \sum_{i=1}^{m} p_i$

$=$ avg of $p_1 \; p_2 \; p_3 \ldots p_m$

RF classification :

- this will include 'm' DT classifier models
- 'm' models generate 'm' predictions

$p_1 \quad p_2 \quad p_3 \ldots\ldots p_m \longrightarrow$ categorical values

final RF prediction : mode of $p_1 \; p_2 \; p_3 \ldots p_m$

Benefits of RF over single DT :

- RF can learn more versatile relationships betn features and target which gives improved performance

more trees $\Rightarrow$ more complex model $\Rightarrow$ more patterns to learn

- Though being a complex model, RF generally avoids overfiting easily

this is due to

1) Bootstrap sampling
- every DT in RF has access to only partial train data due to this sampling
- This avoids any DTree model to go into overfiting

2) Aggregation
- each model has small contribution to final prediction
- contribution from multiple models give overall better generalised predictions

3) Differently learn relationships
- each DTree learns something different than other DTree models which improves the final model prediction.

- RF is more robust to outliers

- RF works really well on high dimensional data

- As DTree, RF are not based on gradient descent or any method which works only on numbers - feature encoding and feature scaling becomes optional in these models.
- RF models usually outperform in many real world use cases.

PCA: Principle component Analysis

- unsupervised ML
- used for dimensionality reduction of data